디지털 논리 회로 설계

[과제리포트]

VHDL 과제

조합논리회로

이름: 원운경

학과: 로봇공학과

학번: 2023010519(원운경)

수업번호: 23278 (장범진 교수님)

실습일: 2026 / 10 / 29

1 실습에 사용한 프로그램

이번 실습에서는 VHDL 로 다섯 가지 조합논리회로를 작성하고 Logisim-evolution 에서 사용할 회로 파일을 구성하였다. 작성한 소스는 Questa 의 테스트 벤치로 검증하였다.

Logisim-evolution

Logisim-evolution 은 입력핀과 출력핀 및 논리 소자를 배치하고 배선하여 회로를 구성하는 프로그램이다.

Questa Altera Starter FPGA Edition 2025.2

Questa 는 VHDL 코드를 컴파일하고 회로의 동작을 시뮬레이션하는 프로그램이다. 이번 검증에서는 입력을 자동으로 바꾸면서 출력이 예상값과 같은지 확인하였다.

2 실습 목적

논리 연산, 입력 선택, 문자 표시, 과반수 판정 및 비트 개수 계산을 각각 VHDL 로 구현하여 조합논리회로의 동작을 이해하는 것이 목적이다.

3 VHDL 코드 및 회로 구성

1 번 4 입력 AND 및 OR 회로

입력 a, b, c, d 와 출력 out1, out2 를 std_logic 자료형으로 선언하였다. out1 에는 네 입력의 AND 결과를 전달하고 out2 에는 OR 결과를 전달하였다. 신호 대입문 두 개는 입력 변화에 따라 동시에 동작한다.

그림 1 Logisim 회로 배치 및 배선

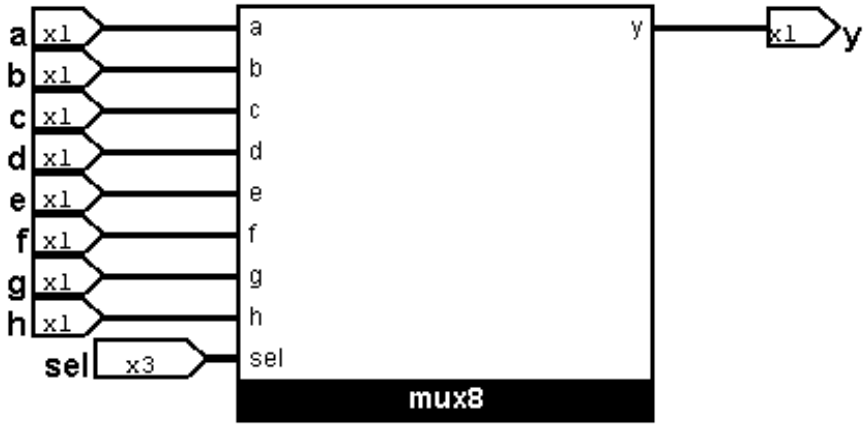


입력 abcd	out1 예상 / 확인	out2 예상 / 확인
0000	0 / 0	0 / 0
0001	0 / 0	1 / 1
1010	0 / 0	1 / 1
1110	0 / 0	1 / 1
1111	1 / 1	1 / 1

2 번 8 대 1 멀티플렉서

멀티플렉서는 여러 데이터 입력 중 하나를 선택하여 출력하는 회로이다. 데이터 입력 a~h 는 각각 1 비트로 두고 선택 입력 sel 은 3 비트 벡터로 선언하였다. with sel select 구문으로 선택값 000~111 을 각 입력에 대응시켰다.

그림 2 Logisim 회로 배치 및 배선

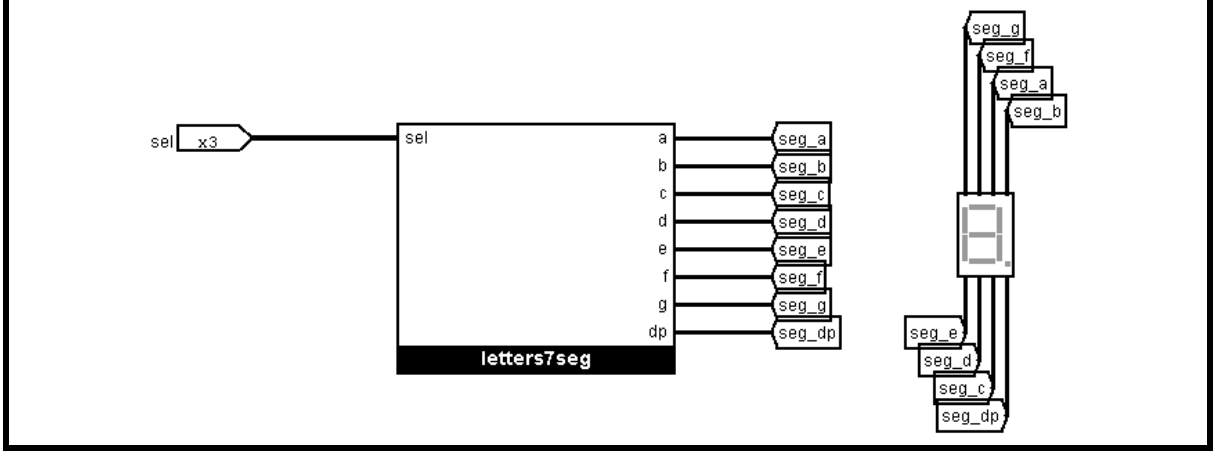


선택 입력 sel	전달되는 입력	선택 입력 sel	전달되는 입력
000	a	100	e
001	b	101	f
010	c	110	g
011	d	111	h

3 번 7 세그먼트 문자 표시

3 비트 선택 입력 sel 에 따라 a, b, c, d, f, g 중 한 문자를 표시하도록 작성하였다. 내부 신호 segments 는 왼쪽부터 abcdefg 순서로 각 막대의 점등 상태를 나타낸다. 각 비트를 별도 출력에 전달하여 표시장치 단자와 직접 연결하였다.

그림 3 Logisim 회로 배치 및 배선



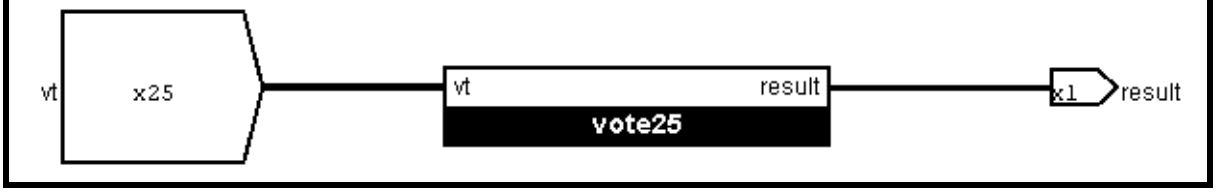
sel	문자	abcdefg	dp
000	a	1111101	0
001	b	0011111	0
010	c	0001101	0
011	d	0111101	0
100	f	1000111	0
101	g	1111011	0
110 / 111	꺼짐	0000000	0

4 번 25 인 투표 회로

입력 vt 는 25 명의 투표를 나타내는 25 비트 벡터이다. 각 비트가 1 이면 찬성, 0 이면 반대로 정의하였다. process 안에서 count 를 0 으로 초기화하고, 반복문으로 각 비트가 1 인지 검사하여 찬성표 수를 계산하였다.

25 명의 과반수는 13 명이므로 count 가 13 이상이면 result 에 1 을 대입하였다. 그보다 작으면 0 을 대입하여 경계값 12 와 13 에서 결과가 달라지도록 구현하였다.

그림 4 Logisim 회로 배치 및 배선

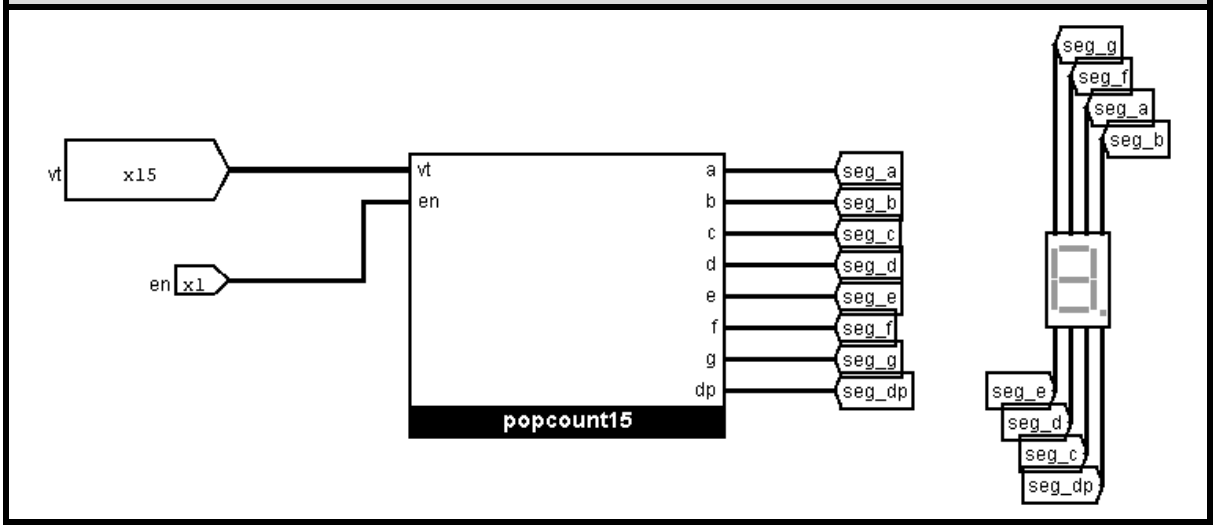


찬성표 수	예상 result	확인 결과
0	0	0
12	0	0
13	1	1
25	1	1

5 번 15 비트 입력의 1 개수 표시

입력 vt 는 표시 범위 0~15 가 모두 발생하도록 15 비트로 구성하였으며 en 은 표시 활성화 신호이다. 반복문으로 vt 에서 1 의 개수를 계산하고 case 구문으로 해당 값을 7 세그먼트 패턴으로 변환하였다. en 이 0 이면 모든 막대를 끄고, 1 이면 계산한 개수를 표시하였다.

그림 5 Logisim 회로 배치 및 배선



en	vt 의 1 개수	표시	abcdefg
0	0~15	꺼짐	0000000
1	0	0	1111110
1	9	9	1111011
1	10	A	1110111
1	11	b	0011111
1	12	C	1001110
1	13	d	0111101
1	14	E	1001111
1	15	F	1000111

4 시뮬레이션 결과 정리

Questa 에서 다섯 모듈과 테스트벤치를 컴파일한 뒤 배치 시뮬레이션을 실행하였다. 컴파일과 시뮬레이션의 종료 코드는 모두 0 이었으며, 마지막 로그에 Errors: 0, Warnings: 0 과 ALL_TESTS_PASSED total_cases=67684 가 기록되었다.

회로	검증 범위	검사 수	결과
logic4	4 비트 전체 입력	16	일치
mux8	데이터 256 가지 × 선택 8 가지	2,048	일치
letters7seg	선택값 8 가지	8	일치
vote25	개수 26 가지 및 경계 위치 50 가지	76	일치
popcount15	15 비트 전체 입력 × en 2 가지	65,536	일치
합계	전체 테스트벤치	67,684	모두 일치

실행 로그

```
PASS logic4 cases=16
PASS mux8 cases=2048
PASS letters7seg cases=8
PASS vote25 cases=76
PASS popcount15 cases=65536
ALL_TESTS_PASSED total_cases=67684
Errors: 0, Warnings: 0
```

이 검증은 VHDL 테스트벤치에서 모듈 출력값을 검사한 결과이다.

부록 VHDL 소스 코드

1 번 logic4.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity logic4 is
    port (
        a    : in  std_logic;
        b    : in  std_logic;
        c    : in  std_logic;
        d    : in  std_logic;
        out1 : out std_logic;
        out2 : out std_logic
    );
end entity logic4;

architecture behavioral of logic4 is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end architecture behavioral;
```

2 번 mux8.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity mux8 is
    port (
        a, b, c, d, e, f, g, h : in  std_logic;
        sel                    : in  std_logic_vector(2 downto 0);
        y                      : out std_logic
    );
end entity mux8;

architecture behavioral of mux8 is
begin
    with sel select
        y <= a   when "000",
             b   when "001",
             c   when "010",
             d   when "011",
             e   when "100",
             f   when "101",
             g   when "110",
             h   when "111",
             '0' when others;
end architecture behavioral;
```

3 번 letters7seg.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity letters7seg is
    port (
        sel                 : in  std_logic_vector(2 downto 0);
        a, b, c, d, e, f, g : out std_logic;
        dp                  : out std_logic
    );
end entity letters7seg;

architecture behavioral of letters7seg is
    signal segments : std_logic_vector(6 downto 0);
begin
    with sel select
        segments <= "1111101" when "000",
                    "0011111" when "001",
                    "0001101" when "010",
                    "0111101" when "011",
                    "1000111" when "100",
                    "1111011" when "101",
                    "0000000" when others;

    a <= segments(6);
    b <= segments(5);
    c <= segments(4);
    d <= segments(3);
    e <= segments(2);
    f <= segments(1);
    g <= segments(0);
    dp <= '0';
end architecture behavioral;
```

4 번 vote25.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity vote25 is
    port (
        vt     : in  std_logic_vector(24 downto 0);
        result : out std_logic
    );
end entity vote25;

architecture behavioral of vote25 is
begin
    process(vt)
        variable count : integer range 0 to 25;
    begin
        count := 0;
        for i in vt'range loop
            if vt(i) = '1' then
                count := count + 1;
            end if;
        end loop;

        if count >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture behavioral;
```

5 번 popcount15.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity popcount15 is
    port (
        vt                  : in  std_logic_vector(14 downto 0);
        en                  : in  std_logic;
        a, b, c, d, e, f, g : out std_logic;
        dp                  : out std_logic
    );
end entity popcount15;

architecture behavioral of popcount15 is
    signal segments : std_logic_vector(6 downto 0);
begin
    process(vt, en)
        variable count : integer range 0 to 15;
    begin
        count := 0;
        for i in vt'range loop
            if vt(i) = '1' then
                count := count + 1;
            end if;
        end loop;

        if en = '1' then
            case count is
                when 0      => segments <= "1111110";
                when 1      => segments <= "0110000";
                when 2      => segments <= "1101101";
                when 3      => segments <= "1111001";
                when 4      => segments <= "0110011";
                when 5      => segments <= "1011011";
                when 6      => segments <= "1011111";
                when 7      => segments <= "1110000";
                when 8      => segments <= "1111111";
                when 9      => segments <= "1111011";
                when 10     => segments <= "1110111";
                when 11     => segments <= "0011111";
                when 12     => segments <= "1001110";
                when 13     => segments <= "0111101";
                when 14     => segments <= "1001111";
                when 15     => segments <= "1000111";
                when others => segments <= "0000000";
            end case;
        else
            segments <= "0000000";
        end if;
    end process;

    a <= segments(6);
    b <= segments(5);
    c <= segments(4);
    d <= segments(3);
    e <= segments(2);
    f <= segments(1);
    g <= segments(0);
    dp <= '0';
end architecture behavioral;
```